

Python3 Decorators

Functions as First-Class Objects

- Functions in Python are treated like regular objects.
- Functions can be assigned to variables.
- Functions can be passed as arguments to other functions.
- Functions can be returned from other functions.
- This capability enables higher-order functions and decorators.

```
def greet():  
    print("Hello")  
message = greet  
message()
```

Higher-Order Functions

- A higher-order function accepts another function as an argument.
- It can also return a function as a result.
- Decorators are implemented using higher-order functions.
- They help add functionality without modifying original code.
- Commonly used for logging, timing, validation, and caching.

```
def execute(func):  
    func()  
def hello():  
    print("Hello")  
execute(hello)
```

Closures

- A closure remembers variables from its enclosing scope.
- The inner function retains access to outer variables.
- Closures exist even after the outer function finishes execution.
- Decorators rely heavily on closures.
- Stored variables can be inspected through **closure**.

```
def outer():  
    message = "Hello"  
  
    def inner():  
        print(message)  
  
    return inner  
  
func = outer()  
  
func()
```

Decorators

- Decorators add functionality to existing functions.
- They wrap another function inside a new function.
- Original function code remains unchanged.
- Decorators use closures internally.
- Applied using the @ syntax.

```
def decorator(func):  
    def wrapper():  
        print("Before")  
        func()  
        print("After")  
  
    return wrapper  
  
@decorator  
  
def greet():
```

```
print("Hello")  
greet()
```

Implementing Function Decorators

- A decorator receives a function as an argument.
- A wrapper function adds extra behavior.
- The wrapper calls the original function.
- The decorator returns the wrapper function.
- The decorated function gets replaced by the wrapper.

```
def email_decorator(func):  
    def wrapper():  
        print("Dear interns,")  
        func()  
        print("Best regards,")  
        print("your new boss")  
    return wrapper  
@email_decorator  
def greeting_message():  
    print("Welcome to your new job!")  
greeting_message()
```

Decorator Execution Flow

- The original function is passed into the decorator.
- The decorator creates a wrapper function.
- The wrapper retains a reference to the original function.
- The original function name points to the wrapper.
- Calling the function executes the wrapper first.

Decorators with Arguments

- Basic decorators only work with functions without parameters.
- *args handles positional arguments.
- **kwargs handles keyword arguments.
- Makes decorators reusable across many functions.
- Preserves flexibility of decorated functions.

```
def logger(func):  
    def wrapper(*args, **kwargs):  
        print("Function called")  
        return func(*args, **kwargs)  
    return wrapper  
  
@logger  
def add(a, b):  
    return a + b  
  
print(add(5, 3))
```

Functools.wraps

- Preserves metadata of the original function.
- Retains function name and docstring.
- Prevents wrapper information from replacing original details.
- Recommended for all custom decorators.
- Imported from functools.

```
from functools import wraps  
  
def logger(func):  
    @wraps(func)  
    def wrapper(*args, **kwargs):  
        print("Running function")
```

```
return func(*args, **kwargs)
```

```
return wrapper
```

Standard Library Decorators

- Python provides many built-in decorators.
- `@classmethod` works with class-level methods.
- `@property` creates computed attributes.
- `@lru_cache` caches function results.
- `@wraps` preserves metadata.

```
from functools import lru_cache
```

```
@lru_cache(maxsize=100)
```

```
def fibonacci(n):
```

```
    if n < 2:
```

```
        return n
```

```
    return fibonacci(n-1) + fibonacci(n-2)
```

`@classmethod`

- Operates on the class instead of an instance.
- Receives `cls` as the first parameter.
- Can access and modify class attributes.
- Useful for alternative constructors.
- Shared across all instances.

```
class Student:
```

```
    count = 0
```

```
    @classmethod
```

```
    def increase_count(cls):
```

```
        cls.count += 1
```

@property

- Converts methods into attributes.
- Improves encapsulation.
- Provides controlled access to data.
- Avoids explicit getter methods.
- Makes code cleaner and more readable.

```
class Person:
    def __init__(self, name):
        self._name = name
    @property
    def name(self):
        return self._name
```

@lru_cache

- Stores results of previous function calls.
- Improves performance of repeated computations.
- Useful for recursive algorithms.
- Reduces unnecessary calculations.
- Implemented using Least Recently Used caching.

```
from functools import lru_cache
@lru_cache()
def square(n):
    print("Calculating...")
    return n * n
```

Timing Decorator

- Measures function execution time.
- Useful for performance analysis.

- Common real-world decorator example.
- Helps identify slow functions.
- Frequently used during optimization.

```
import time

from functools import wraps

def timer(func):

    @wraps(func)

    def wrapper(*args, **kwargs):

        start = time.time()

        result = func(*args, **kwargs)

        end = time.time()

        print("Execution Time:", end - start)

        return result

    return wrapper
```

Logging Decorator

- Records function execution details.
- Useful for debugging and monitoring.
- Tracks function calls automatically.
- Can log parameters and results.
- Common in production systems.

```
from functools import wraps

def logger(func):

    @wraps(func)

    def wrapper(*args, **kwargs):

        print(f"Calling {func.__name__}")

        return func(*args, **kwargs)

    return wrapper
```

Decorator Benefits

- Promotes code reuse.
- Keeps business logic clean.
- Separates concerns effectively.
- Reduces duplicate code.
- Simplifies maintenance.

Decorator Workflow Summary

- Functions are first-class objects.
- Higher-order functions enable decorators.
- Closures retain nonlocal variables.
- Decorators wrap functions with additional functionality.
- `functools.wraps` preserves original metadata.